Practical No.: 9

Title: Demonstrating the Use of GET Method in ReactJS & NodeJS

Aim:

To demonstrate how to **fetch data from a Node.js backend to a React frontend** using the **GET method** with the `fetch` API.

Theory:

What is the GET Method?

- The **GET method** is used to **retrieve data** from the server.
- It does not modify or send data, only **requests** and **displays** information.
- Commonly used to fetch **user details, lists, and API responses**.

Steps Involved:

1. Create a **Node.js backend** that sends data on a GET request.
2. Use **fetch API** in React to retrieve and display the data.

Syntax of GET Request in React:

```
fetch("http://localhost:5000/data")
  .then(response => response.json())
  .then(data => console.log(data));
```

ReactJS Code

Step 1: Create a React Component (`index.html` & `get_request.js`)

HTML File (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>React GET Method</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
    <script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
</head>
<body>
    <h2>Practical 9: GET Method in React</h2>
    <div id="root"></div>

    <script type="text/babel" src="get_request.js"></script>
</body>
</html>
```

ReactJS File (get_request.js)

```
function App() {
    const [data, setData] = React.useState([]);

    React.useEffect(() => {
        fetch("http://localhost:5000/data")
            .then(response => response.json())
            .then(data => setData(data));
    }, []);

    return (
        <div>
            <h3>Fetched Data from Server</h3>
            <ul>
                {data.map((item, index) => (
                    <li key={index}>{item}</li>
                ))}
            </ul>
        </div>
    );
}

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

Step 2: Create a Node.js Backend (server.js)

```
const express = require("express");
const cors = require("cors");
const app = express();

app.use(cors()); // Enable CORS for frontend-backend communication

app.get("/data", (req, res) => {
    const sampleData = ["ReactJS", "NodeJS", "Express", "MongoDB"];
    res.json(sampleData);
});

app.listen(5000, () => console.log("Server running on port 5000"));
```

Steps to Run the Application:

1. **Start the Backend:**
   - Install dependencies: `npm install express cors`
   - Run the server: `node server.js`
2. **Open `index.html` in the browser** to see the fetched data.